Metodologie di Programmazione (A-L)

II semestre - a.a. 2023 – 2024

Parte 8 – Strutture Dati**

a cura di Mattia Samory*

DIPARTIMENTO
DI INFORMATICA

SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

📣Avvisi

- ci sarà lezione lunedì 29/4
- tutoraggio
- versioni Horstmann (Big Java. Early Objects 7ed)

Letture

- Strutture dati e Java Collections Framework (Horstmann ITA 14)

🔄 Recap

- programmazione generica

- I tipi generici astraggono altri tipi, garantendo la correttezza della tipizzazione nel programma a tempo di compilazione.
- Quando viene invocata una dichiarazione generica, i tipi formali vengono sostituiti da quelli effettivi

```
public class ArrayList<T>
```

tipo formale

tipo effettivo

```
ArrayList<Integer> list1 = new ArrayList<Integer>;
ArrayList<GoTCharacter> list2 = new ArrayList<GoTCharacter>;
```

🎯Obiettivi

- Java Collections Framework
- iterazione su Collection
- ADT & Collection Zoo: *List, *Set, *Map
- Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
- Riapplichiamo ADT & Collection: Pila e Coda
- Scegliere una struttura dati
- Alberi

A cosa serve una struttura dati?





A cosa serve una struttura dati?

- A memorizzare e organizzare i dati in memoria così da utilizzarli in modo efficiente



Una mano di carte



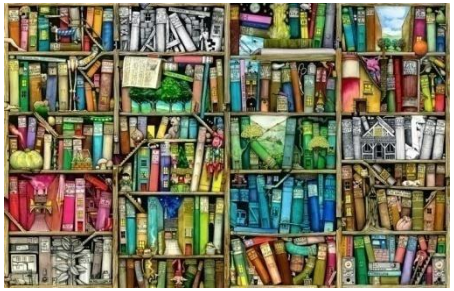
Una sequenza di canzoni



Una squadra di pallavolo



Uno spartito



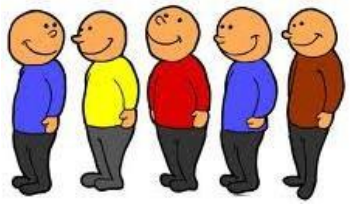
Una libreria

Caratteristiche di una struttura dati

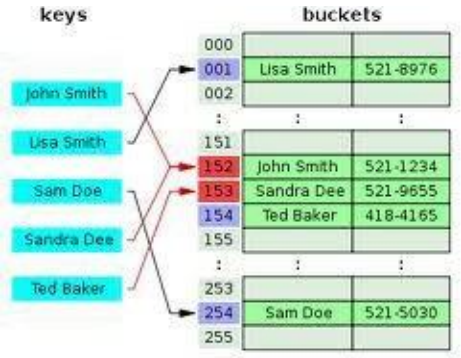
- E’ necessario mantenere un ordine?

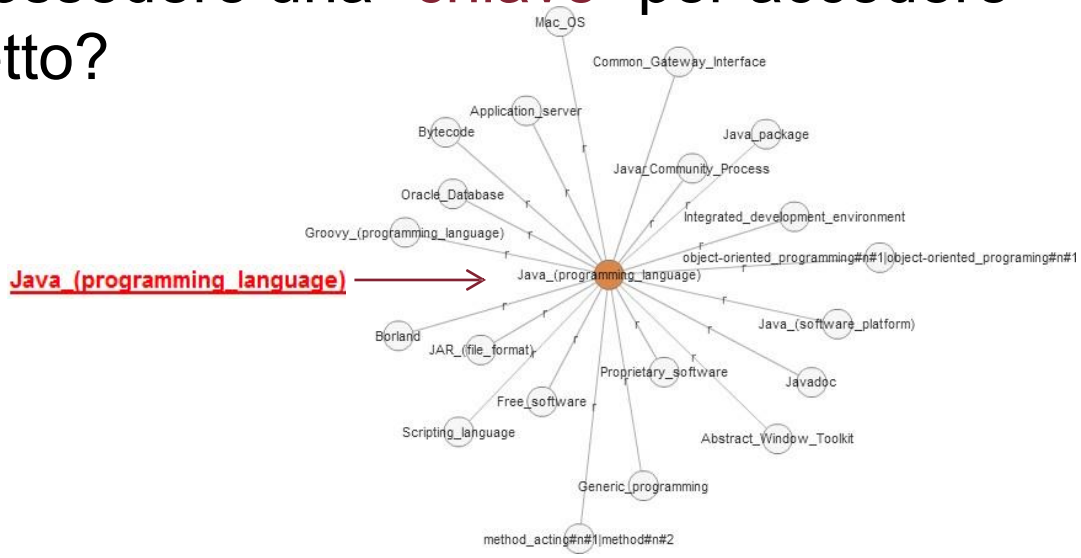


- Gli oggetti nella struttura possono ripetersi?



- E’ utile/necessario possedere una “chiave” per accedere a uno specifico oggetto?





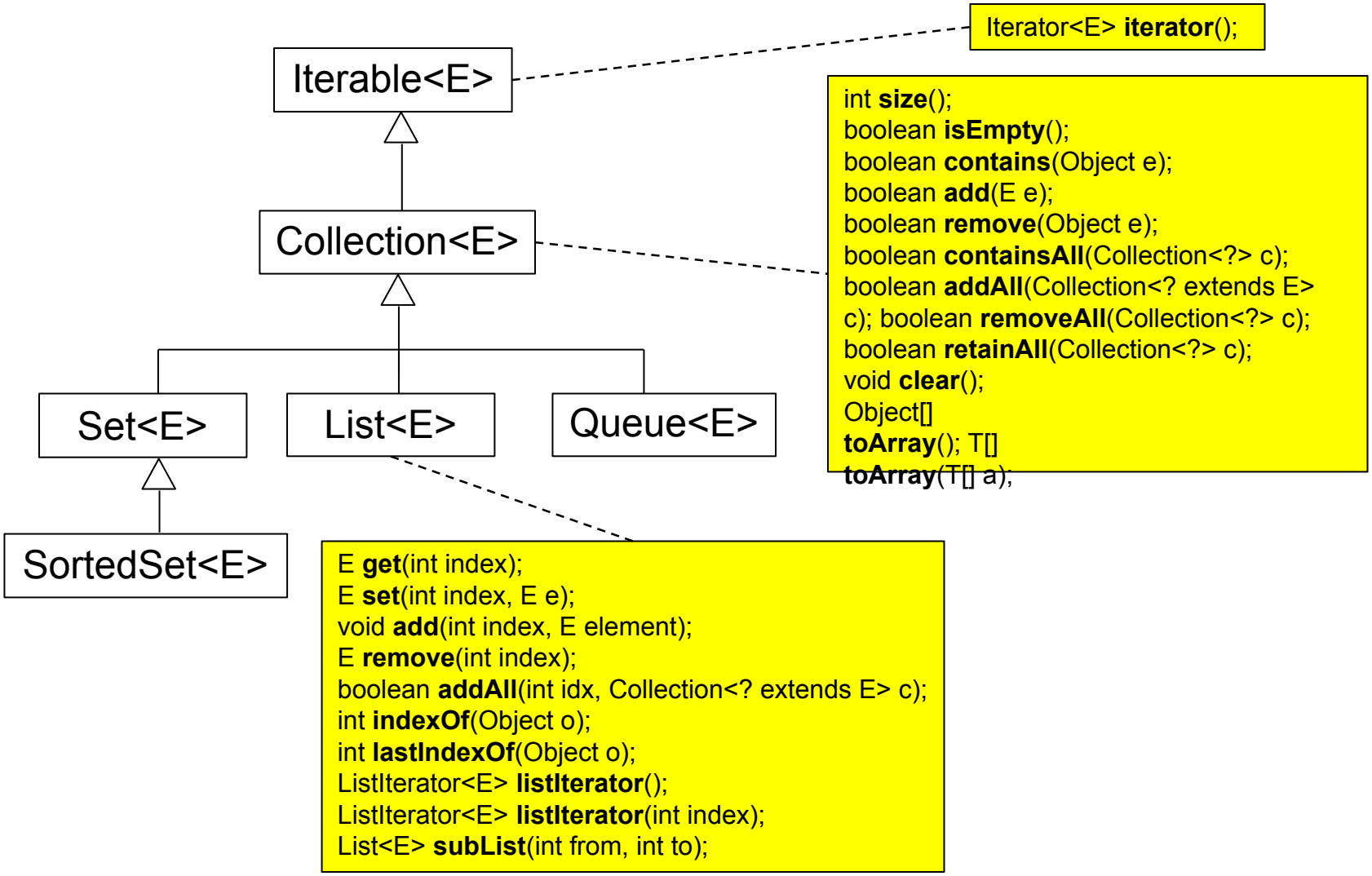
Abstract Data Type (ADT)	**Struttura dati**
Definisce un insieme di operazioni che si possono applicare ai dati, senza descrivere come sono implementate.	Implementa un ADT in un linguaggio di programmazione.
Esempio: Una lista è un ADT lineare in cui gli elementi sono organizzati in ordine sequenziale. *Operazioni:* Inserimento, cancellazione, accesso, attraversamento, etc.	(1) Una lista implementata con **array** (2) Una lista implemetata con **singly-linked list**
Teorica. Il “cosa.”	Concreta. Il “come.”
In Java: Interfacce	In Java: classi che implementano interfacce

Le Collection

- Le collezioni in Java sono rese disponibili mediante il framework delle collezioni (Java Collection Framework)
- Strutture dati già pronte all'uso
  - con interfacce e algoritmi per manipolarle
- Contengono e "strutturano" **riferimenti** ad altri oggetti
  - Tipicamente tutti "dello stesso tipo"
- Alcune interfacce del framework:

Interfaccia	Descrizione
Collection	L'interfaccia alla radice della gerarchia di collezioni
Set	Una collezione senza duplicati
List	Una collezione ordinata che può contenere duplicati
Map	Associa coppie di (chiave, valore), senza chiavi duplicate
Queue	Una collezione first-in, first-out che modella una coda

Gerarchia delle interfacce di tipo Collection



Collezioni fondamentali



- Saranno molto utili nel tempo:
  - ArrayList
  - LinkedList
    - Estendono entrambe AbstractList
  - HashSet
  - TreeSet
    - Estendono entrambe AbstractSet
  - LinkedHashSet
    - Estende HashSet
  - HashMap
  - TreeMap
    - Estendono AbstractMap
  - LinkedHashMap
    - Estende HashMap

🎯Obiettivi

- ✅ Java Collections Framework
  - iterazione su Collection
  - ADT & Collection Zoo: *List, *Set, *Map
  - Utilità ed algoritmi per JCF
  - Riferimenti a metodi
  - Comparator
  - Interfacce funzionali built-in
  - Riapplichiamo ADT & Collection: Pila e Coda
  - Scegliere una struttura dati
  - Alberi

Iterazione *esterna* su una collezione

1. Mediante gli Iterator:

```
// "vecchio" stile, ma maggiore controllo
Iterator<Integer> i = collezione.iterator();
while(i.hasNext())
{
    int k = i.next();
    System.out.println(k);
}
```

Finché c'è ancora un elemento

Ottieni il prossimo elemento

2. Mediante il costrutto "for each":

```
// "nuovo" stile: for each, più elegante
for (Integer k : collezione)
    System.out.println(k);
```

Per ogni elemento k nella collezione

3. Mediante indici (solo su liste):

```
// su collezioni che implementano List: elemento per elemento
for (int j = 0; j < collezione.size(); j++)
{
    int k = collezione.get(j);
    System.out.println(k);
}
```

Accesso elemento per elemento

Iterazione *interna* su una collezione

- Mediante il metodo Iterable.forEach permette l'iterazione su qualsiasi collezione senza specificare come effettuare l'iterazione
  - Utilizza il polimorfismo (chiamerà il forEach della classe specifica)
- forEach prende in input un Consumer, che è un'**interfaccia funzionale** con un solo metodo:

  void accept(T t);

- Esempio:

```
List<Integer> l = List.of(4, 8, 15, 16, 23,
42); l.forEach(x -> System.out.println(x));
```

Modificare una lista durante l'iterazione

- Non è possibile utilizzare metodi di modifica durante un'iterazione:

```
for (int k : l)
    if (k == x) { l.remove(x); }
```

```
Exception in thread "main" java.util.ConcurrentModificationException
        at java.util.ArrayList$Itr.checkForComodification(Unknown Source)
        at java.util.ArrayList$Itr.next(Unknown Source)
        at ErroriComuni.main(ErroriComuni.java:27)
```

- ma è possibile utilizzare Iterator.remove:

```
Iterator<Integer> i = l.iterator();
while(i.hasNext())
    if (i.next() == x) i.remove();
```

🎯Obiettivi

- ✅ Java Collections Framework
- ✅ iterazione su Collection
  - ADT & Collection Zoo: *List, *Set, *Map
  - Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
  - Riapplichiamo ADT & Collection: Pila e Coda
  - Scegliere una struttura dati
  - Alberi

Le liste: ArrayList e LinkedList



- Basate su List, una sottointerfaccia di Collection e di Iterable
- Le classi liste sono: ArrayList e LinkedList
  - estendono AbstractList e implementano l'interfaccia List
- ArrayList implementa la lista mediante un array (eventualmente ridimensionato)
  - Capacità iniziale: 10 elementi
- LinkedList implementa la lista mediante elementi linkati

Metodi di ArrayList



Method Summary	
boolean	`add(E e)` Appends the specified element to the end of this list.
void	`add(int index, E element)` Inserts the specified element at the specified position in this list.
boolean	`addAll(Collection<? extends E> c)` Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
boolean	`addAll(int index, Collection<? extends E> c)` Inserts all of the elements in the specified collection into this list, starting at the specified position.
void	`clear()` Removes all of the elements from this list.
Object	`clone()` Returns a shallow copy of this `ArrayList` instance.
boolean	`contains(Object o)` Returns `true` if this list contains the specified element.
void	`ensureCapacity(int minCapacity)` Increases the capacity of this `ArrayList` instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
E	`get(int index)` Returns the element at the specified position in this list.
int	`indexOf(Object o)` Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	`isEmpty()` Returns `true` if this list contains no elements.
int	`lastIndexOf(Object o)` Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
E	`remove(int index)` Removes the element at the specified position in this list.
boolean	`remove(Object o)` Removes the first occurrence of the specified element from this list, if it is present.
protected void	`removeRange(int fromIndex, int toIndex)` Removes from this list all of the elements whose index is between `fromIndex`, inclusive, and `toIndex`, exclusive.
E	`set(int index, E element)` Replaces the element at the specified position in this list with the specified element.
int	`size()` Returns the number of elements in this list.
Object[]	`toArray()` Returns an array containing all of the elements in this list in proper sequence (from first to last element).
<T> T[]	`toArray(T[] a)` Returns an array containing all of the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.
void	`trimToSize()` Trims the capacity of this `ArrayList` instance to be the list's current size.

Alcuni metodi aggiuntivi di LinkedList



Metodo	Descrizione
void **addFirst**(E e)	Aggiunge l'elemento in testa alla lista
void **addLast**(E e)	Aggiunge l'elemento in coda alla lista
Iterator<E> **descendingIterator**()	Restituisce un iteratore che parte dall'ultimo elemento della lista e si sposta verso sinistra
E **getFirst**()	Restituisce il primo elemento della lista
E **getLast**()	Restituisce l'ultimo elemento della lista
E **removeFirst**()	Rimuove e restituisce il primo elemento
E **removeLast**()	Rimuove e restituisce l'ultimo elemento
E **pop**()	Elimina e restituisce l'elemento in cima alla lista vista come pila
void **push**(E e)	Inserisce un elemento in cima alla lista vista come pila

Iterare sulle liste in entrambe le direzioni

- listIterator() restituisce un iteratore bidirezionale (interfaccia ListIterator che estende Iterator) per la lista
  - Da sinistra verso destra:

```
ListIterator<Integer> i = l.listIterator();
while(i.hasNext())
    System.out.println(i.next());
```

  - Da destra verso sinistra (se si specifica un intero, si parte da quella posizione: es. list.listIterator(list.size()))

```
ListIterator<Integer> i = l.listIterator(l.size());
while(i.hasPrevious())
    System.out.println(i.previous());
```

ListIterator

Method Summary	
void	**add**(E e) Inserts the specified element into the list (optional operation).
boolean	**hasNext**() Returns `true` if this list iterator has more elements when traversing the list in the forward direction.
boolean	**hasPrevious**() Returns `true` if this list iterator has more elements when traversing the list in the reverse direction.
E	**next**() Returns the next element in the list.
int	**nextIndex**() Returns the index of the element that would be returned by a subsequent call to `next`.
E	**previous**() Returns the previous element in the list.
int	**previousIndex**() Returns the index of the element that would be returned by a subsequent call to `previous`.
void	**remove**() Removes from the list the last element that was returned by `next` or `previous` (optional operation).
void	**set**(E e) Replaces the last element returned by `next` or `previous` with the specified element (optional operation).

Insiemi: HashSet, TreeSet e LinkedHashSet

- Basati su Set, una sottointerfaccia di Collection e di Iterable
- Gli insiemi sono Collection che contengono elementi **tutti distinti**
- Le classi insiemi, HashSet, TreeSet e LinkedHashSet, estendono AbstractSet e implementano l'interfaccia Set
- HashSet memorizza gli elementi in una tabella di hash
- TreeSet memorizza gli elementi in un albero mantenendo un ordine sugli elementi
- LinkedHashSet memorizza gli elementi in ordine di inserimento

HashSet: Un esempio

- Salviamo nomi e cognomi in due insiemi:

```
HashSet<String> nomi = new HashSet<String>();
HashSet<String> cognomi = new HashSet<String>();

nomi.add("mario");
cognomi.add("rossi");

nomi.add("mario");
cognomi.add("bianchi");

nomi.add("mario");
cognomi.add("verdi");

nomi.add("luigi");
cognomi.add("rossi");

nomi.add("luigi");
cognomi.add("bianchi");

System.out.println(nomi);
System.out.println(cognomi);
```

- Stampandoli, ottieniamo:

```
[mario, luigi]
[verdi, bianchi, rossi]
```

TreeSet: Un esempio

- Salviamo nomi e cognomi in due insiemi:

```
TreeSet<String> nomi = new TreeSet<String>();
TreeSet<String> cognomi = new TreeSet<String>();

nomi.add("mario");
cognomi.add("rossi");

nomi.add("mario");
cognomi.add("bianchi");

nomi.add("mario");
cognomi.add("verdi");

nomi.add("luigi");
cognomi.add("rossi");

nomi.add("luigi");
cognomi.add("bianchi");

System.out.println(nomi);
System.out.println(cognomi);
```

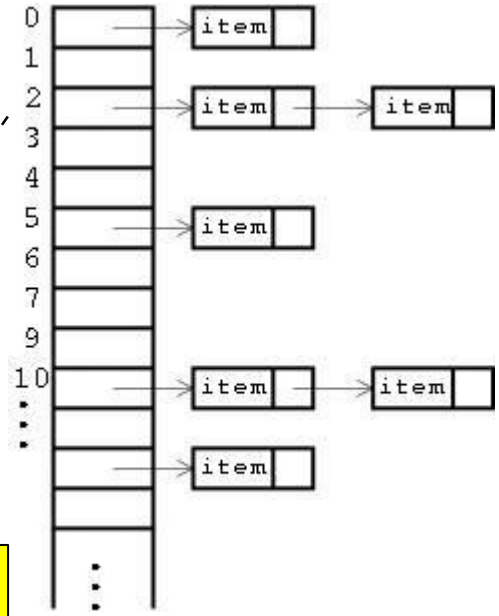
- Stampandoli, ottieniamo:

```
[luigi, mario]
[bianchi, rossi, verdi]
```

Ordinati secondo l'ordinamento naturale del tipo (String)

Come funziona un HashSet?

- Si fonda sul concetto di tabella hash



Questo numero è una funzione dell'hashCode() dell'oggetto usato come chiave



Esercizio

- Si definisca una classe Canzone i cui oggetti sono costruiti con nome e autore della canzone
- Si crei una classe Raccolta che contiene (ovvero i cui oggetti sono costruiti con) un insieme di canzoni
  - In una prima versione, si utilizzi HashSet per memorizzare l'insieme di canzoni
  - Si implementi correttamente equals e hashCode in modo da evitare duplicati di canzoni sostanzialmente uguali (ovvero con lo stesso nome e autore)
  - La classe è dotata di un metodo toString che restituisce la rappresentazione sotto forma di stringa dell'insieme di canzoni (analogamente, le canzoni sono dotato dello stesso metodo)
  - Si implementi quindi la classe RaccoltaOrdinata che utilizza TreeSet per mantenere un ordine sulle canzoni
  - Implementare un ordine sulle canzoni basato sul nome della canzone e, se questo è uguale, sul nome dell'autore

Mappe

- Una mappa mette in corrispondenza chiavi e valori
- **Non** può contenere chiavi duplicate
- java.util.Map è un'interfaccia implementata da HashMap, TreeMap e LinkedHashMap

L'interfaccia java.util.Map

Method Summary

Methods

Modifier and Type	Method and Description
void	`clear()` Removes all of the mappings from this map (optional operation).
boolean	`containsKey(Object key)` Returns true if this map contains a mapping for the specified key.
boolean	`containsValue(Object value)` Returns true if this map maps one or more keys to the specified value.
`Set<Map.Entry<K,V>>`	`entrySet()` Returns a `Set` view of the mappings contained in this map.
boolean	`equals(Object o)` Compares the specified object with this map for equality.
`V`	`get(Object key)` Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
int	`hashCode()` Returns the hash code value for this map.
boolean	`isEmpty()` Returns true if this map contains no key-value mappings.
`Set<K>`	`keySet()` Returns a `Set` view of the keys contained in this map.
`V`	`put(K key, V value)` Associates the specified value with the specified key in this map (optional operation).
void	`putAll(Map<? extends K,? extends V> m)` Copies all of the mappings from the specified map to this map (optional operation).
`V`	`remove(Object key)` Removes the mapping for a key from this map if it is present (optional operation).
int	`size()` Returns the number of key-value mappings in this map.
`Collection<V>`	`values()` Returns a `Collection` view of the values contained in this map.

Mappe

- Una mappa mette in corrispondenza chiavi e valori
- **Non** può contenere chiavi duplicate
- java.util.Map è un'interfaccia implementata da HashMap, LinkedHashMap e TreeMap
- HashMap memorizza le coppie in una tabella di hash
- TreeMap memorizza le coppie in un albero mantenendo un ordine sulle chiavi
- LinkedHashMap estende HashMap e mantiene l'ordinamento di iterazione secondo gli inserimenti effettuati

HashMap: Un esempio

- Registriamo le frequenze di occorrenza delle parole in un testo:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new HashMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)   frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

TreeMap: Un esempio

- Se vogliamo mantenere un ordinamento sulle chiavi (ovvero sulle parole) di cui calcoliamo la frequenza:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new TreeMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)   frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

TreeMap mantiene l'ordinamento delle chiavi

LinkedHashMap: Un esempio

- Se vogliamo mantenere un ordinamento di iterazione (ovvero sulle parole) di cui calcoliamo la frequenza:

```
public class MappaDelleFrequenze
{
    private Map<String, Integer> frequenze = new LinkedHashMap<String, Integer>();

    public MappaDelleFrequenze(File file) throws IOException
    {
        Scanner in = new Scanner(file);
        while(in.hasNext())
        {
            String parola = in.next();
            Integer freq = frequenze.get(parola);

            // parola mai incontrata (non presente nella mappa)
            if (freq == null)   frequenze.put(parola, 1);
            // incrementa il conteggio di frequenza della parola
            else                frequenze.put(parola, freq+1);
        }
    }
}
```

LinkedHashMap mantiene l'ordinamento di iterazione sulle chiavi

Confronto tra HashMap, TreeMap e LinkedHashMap

- Dato il seguente file:

```
a a e b c d e e b
```

- Se iteriamo su ciascuna delle tre strutture:

```
String s = "{";

for (String chiave : frequenze.keySet())
    s += chiave+"="+frequenze.get(chiave)+", ";

s += "}";
```

otteniamo:

- HashMap: {d=1, e=3, b=2, c=1, a=2, }
- TreeMap: {a=2, b=2, c=1, d=1, e=3, }
- LinkedHashMap: {a=2, e=3, b=2, c=1, d=1, }

Senza ordinamento evidente

Ordinamento naturale

Ordinamento di inserimento

Metodi dell'interfaccia Map

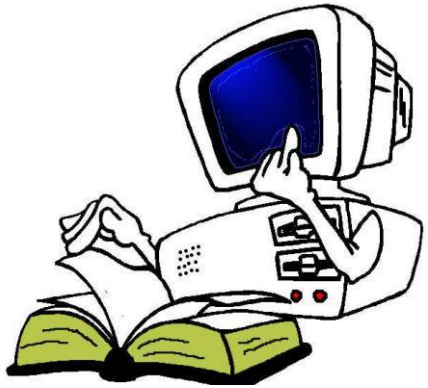
Method Summary

Methods

Modifier and Type	Method and Description
void	`clear()` Removes all of the mappings from this map (optional operation).
boolean	`containsKey(Object key)` Returns true if this map contains a mapping for the specified key.
boolean	`containsValue(Object value)` Returns true if this map maps one or more keys to the specified value.
`Set<Map.Entry<K,V>>`	`entrySet()` Returns a `Set` view of the mappings contained in this map.
boolean	`equals(Object o)` Compares the specified object with this map for equality.
`V`	`get(Object key)` Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.
int	`hashCode()` Returns the hash code value for this map.
boolean	`isEmpty()` Returns true if this map contains no key-value mappings.
`Set<K>`	`keySet()` Returns a `Set` view of the keys contained in this map.
`V`	`put(K key, V value)` Associates the specified value with the specified key in this map (optional operation).
void	`putAll(Map<? extends K,? extends V> m)` Copies all of the mappings from the specified map to this map (optional operation).
`V`	`remove(Object key)` Removes the mapping for a key from this map if it is present (optional operation).
int	`size()` Returns the number of key-value mappings in this map.
`Collection<V>`	`values()` Returns a `Collection` view of the values contained in this map.

Chiavi e valori di una mappa

- E' possibile ottenere l'insieme delle chiavi di una mappa mediante il metodo keySet
- L'elenco dei valori mediante il metodo values **(con ripetizione!)**
- L'insieme delle coppie (chiave, valore) mediante il metodo entrySet
  - Restituisce un insieme di oggetti di tipo Map.Entry<K, V>
  - Per ogni oggetto (ovvero la coppia) è possibile conoscere la chiave (getKey()) e il valore (getValue())



Un esempio di creazione di una TreeMap

- Il seguente codice crea una TreeMap e ne stampa l'insieme delle chiavi, la collezione dei valori e la collezione delle coppie (di tipo Map.Entry<K, V>):

```
Map<String, Integer> m = new TreeMap<String, Integer>();

m.put("a", 1);
m.put("b", 1);
m.put("c", 2);
m.put("d", 3);
m.put("e", 1);

System.out.println(m.keySet());
System.out.println(m.values());
System.out.println(m.entrySet());
```

- Output:

```
[a, b, c, d, e]
[1, 1, 2, 3, 1]
[a=1, b=1, c=2, d=3, e=1]
```

Alcuni metodi Java 8 e Java 9 nell'interfaccia java.util.Map

- forEach(BiConsumer)
  - Itera su ciascuna coppia (chiave, valore)
- getOrDefault(chiave, valoreDefault)
  - Restituisce il valore associato alla chiave o valoreDefault se la chiave non è presente
- merge(chiave, valore, BiFunction)
  - Se la chiave non contiene già un valore, imposta il valore specificato, altrimenti chiama una bifunzione che decide come mettere insieme il valore precedente con il valore passato in input
- *of*(chiave, valore, chiave, valore, …, chiave, valore)
  - statico, crea una mappa immutabile dei tipi e con i valori corrispondenti



Esercizio: multimappa

- Creare una multimappa (ovvero una mappa che permette, a fronte della stessa chiave, di memorizzare più valori) esponendo i metodi:
- get e put
  - Si faccia uso dei metodi più avanzati disponibili in Java 8 (ad es. getOrDefault o merge)
- keySet e values (quest'ultimo restituisce l'elenco di tutti i valori presenti nella mappa, anche con ripetizione)
- una versione di values che restituisce una lista di elementi ordinata secondo una comparazione ad hoc
- valueSet che restituisce l'insieme di tutti i valori contenuti nella multimappa

🎯Obiettivi

- ✅ Java Collections Framework
- ✅ iterazione su Collection
- ✅ ADT & Collection Zoo: *List, *Set, *Map
  - Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
  - Riapplichiamo ADT & Collection: Pila e Coda
  - Scegliere una struttura dati
  - Alberi

Algoritmi sulle collezioni: la classe java.util.Collections

- Fornisce metodi statici per la manipolazione delle collezioni:

Metodo	Descrizione
sort	Ordina gli elementi di una List
binarySearch	Cerca un elemento di una List mediante ricerca binaria
fill	Rimpiazza tutti gli elementi di una List con l'elemento specificato
copy	Copia gli elementi da una lista all'altra
reverse	Inverte l'ordine degli elementi di una List
shuffle	Mette in ordine casuale gli elementi di una List
min/max	Restituisce l'elemento più piccolo/grande della Collection

Algoritmi sugli array: la classe java.util.Arrays

- Fornisce metodi statici per la manipolazione degli array:

Metodo	Descrizione
sort	Ordina gli elementi dell'array
binarySearch	Cerca un elemento mediante ricerca binaria
fill	Riempi l'array con l'elemento specificato
copyOf	Restituisce una copia di un array
equals	Confronta due array elemento per elemento
asList	Restituisce una lista contenente gli elementi dell'array
toString	Restituisce una rappresentazione dell'array sotto forma di stringa

Ordinamento “naturale”

- Come garantire un ordinamento sui tipi utilizzati nelle strutture dati che si fondano su un ordinamento (es. TreeSet o TreeMap)?
- E’ necessario che quei tipi implementino un’interfaccia speciale, chiamata Comparable<T>
- Dotata di un solo metodo:

Metodo	Descrizione
int **compareTo**(T o)	Confronta sé stesso con l’oggetto o (restituendo 0 se uguali, -1 se <= o, +1 altrimenti)

L'interfaccia Comparable: un esempio

- Come garantire l'ordinamento di una classe NomeCognome?

```
public class NomeCognome implements Comparable<NomeCognome>
{
    private String nome;
    private String cognome;

    public NomeCognome(String nome, String cognome)
    {
        this.nome = nome;
        this.cognome = cognome;
    }

    @Override
    public int compareTo(NomeCognome o)
    {
        int v = cognome.compareTo(o.cognome);

        if (v == 0) return nome.compareTo(o.nome);
        else return v;
    }

    @Override
    public String toString() { return cognome+" "+nome; }
}
```

Ordinamento con l'interfaccia Comparator<T>

- E se la classe non fornisce "di suo" l'ordinamento mediante l'interfaccia Comparable?
- E se voglio ordinare diversamente gli elementi di un certo oggetto?
- E' sufficiente implementare un'interfaccia Comparator<T> e passarne un'istanza in input al costruttore delle strutture dati (es. TreeSet e TreeMap)
- Metodi dell'intefaccia Comparator<T>

Method Summary	
int	compare(T o1, T o2) Compares its two arguments for order.
boolean	equals(Object obj) Indicates whether some other object is "equal to" this comparator.

Riferimenti a metodi esistenti

- E' possibile passare riferimenti a metodi esistenti
- Utilizzando la sintassi:
  - Classe::metodoStatico
  - riferimentoOggetto::metodoNonStatico
  - Classe::metodoNonStatico

- Esempio:

```
Converter<String, Integer> converter = Integer::valueOf;
Integer converted = converter.convert("123");
```

Riferimento a metodi d'istanza usando il nome della classe vs. usando un riferimento a un oggetto

- Che differenza c'è tra:
  - riferimentoOggetto::metodo e
  - nomeClasse::metodo?
- Nel primo caso, il metodo sarà chiamato sull'oggetto riferito
- Nel secondo caso, non stiamo specificando su quale oggetto applicare il metodo
  - Ma il metodo è d'istanza, quindi utilizza membri d'istanza (campi, metodi, ecc.)
  - Ci si riferisce al metodo **implicitamente esteso** con un primo parametro aggiuntivo: un riferimento a un oggetto della classe cui appartiene il metodo



Esempio di riferimento a metodi d'istanza mediante classe

- Considerate Arrays.sort(T[] a, Comparator<? super T> c)
- E Comparator:

```
@FunctionalInterface
public interface Comparator<T>
{
    int compare(T o1, T o2);
    boolean equals(Object o);
}
```

- Posso scrivere:

```
Arrays.sort(new String[] { "a", "c", "b" }, String::compareTo);
```



Esempio di riferimento a metodi d'istanza mediante classe

```
@FunctionalInterface
public interface StringProcessor
{
    String process(String s);
}

// riferimento a metodo d'istanza String toLowerCase()
StringProcessor f = String::toLowerCase;
System.out.println(f.process("BELLA"));  // stampa “bella”
String s = "bella";
// riferimento a metodo d'istanza String concat(String s)
StringProcessor g = s::concat;  // stesso di: “bella”::concat
System.out.println(g.process(" zi'!")); // stampa “bella zi'!”
// riferimento a metodo statico String valueOf(Object o)
StringProcessor h = String::valueOf;
System.out.println(h.process("bella"));    // stampa
“bella”
```

Visibilità dalle espressioni lambda

- Accesso alla variabili esterne da un'espressione lambda simile a quello di una classe anonima
- Si possono accedere:
  - Campi d'istanza e variabili statiche
  - Variabili **final** del metodo che definisce la lambda

- Esempio:

```
final int num = 1;
Converter<Integer, String> stringConverter =
                                from -> String.valueOf(from + num);
stringConverter.convert(2); // "3"
```

Visibilità dalle espressioni lambda

- Accesso alla variabili esterne da un'espressione lambda simile a quello di una classe anonima
- Si possono accedere:
  - Campi d'istanza e variabili statiche
  - Variabili **final** del metodo che definisce la lambda
  - Variabili del metodo esterno **implicitamente final**
- Esempio:

```
int num = 1; // non viene modificata all'interno del metodo
Converter<Integer, String> stringConverter =
                                    from -> String.valueOf(from + num);
stringConverter.convert(2); // "3"
```



Ordinamento inverso: da Java 7 a Java 8

In Java 7:

```
List<String> names = Arrays.asList("peter", "anna", "mike", "xenia");
Collections.sort(names, new Comparator<String>()
{
    @Override
    public int compare(String a, String b) { return b.compareTo(a); }
});
```

In Java 8:

```
Collections.sort(names, (a, b) -> b.compareTo(a));
```



Ordinamento per lunghezza: da Java 7 a Java 8

In Java 7:

```
List<String> names = Arrays.asList("peter", "anna", "mike", "xenia");
Collections.sort(names, new Comparator<String>()
{
    @Override
    public int compare(String a, String b)
    {
        return Integer.compare(a.length(), b.length());
    }
});
```

In Java 8:

```
Collections.sort(names, (a, b) -> a.length()-b.length());
                    // oppure: (a, b) -> Integer.compare(a.length(), b.length());
```



Esempio

```
class Person
{
        private String firstName;
        private String lastName;

        public Person() {} // non fatelo a casa!!!
        public Person(String firstName, String lastName)
        {
                this.firstName = firstName;
                this.lastName = lastName;
        }
}
```

Metodi di default nell'interfaccia Comparator

```
Comparator<Person> comparator = (p1, p2) -> p1.firstName.compareTo(p2.firstName);
Person p1 = new Person("John", "Doe");
Person p2 = new Person("Alice", "Wonderland");
comparator.compare(p1, p2); // > 0
```

- Confronto inverso:

```
comparator.reversed().compare(p1, p2); // < 0
```

- Confronto per una data chiave specificata:

```
comparator = Comparator.comparing(p -> p.getFirstName());
```

- Confronto per criteri multipli a cascata:

```
comparator = Comparator.comparing(p -> p.getFirstName())
        .thenComparing(p -> p.getLastName())
```

Metodi di default nell'interfaccia Comparator

```
Comparator<Person> comparator = (p1, p2) -> p1.firstName.compareTo(p2.firstName);
Person p1 = new Person("John", "Doe");
Person p2 = new Person("Alice", "Wonderland");
comparator.compare(p1, p2); // > 0
```

- Confronto inverso:

```
comparator.reversed().compare(p1, p2); // < 0
```

- Confronto per una data chiave specificata:

```
comparator = Comparator.comparing(p -> p.getFirstName());
```

- Confronto per criteri multipli a cascata:

```
comparator = Comparator.comparing(Person::getFirstName)
        .thenComparing(Person::getLastName)
```

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione a valori booleani a un solo argomento generico T

```
Predicate<String> predicate = s -> s.length() > 0;
predicate.test("foo"); // true
predicate.test(""); // false
```

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione a valori booleani a un solo argomento generico T

```
Predicate<String> predicate = s -> s.length() > 0;
predicate.test("foo"); // true
predicate.negate().test("foo"); // false

Predicate<String> predicate2 = s -> s.startsWith("f");

predicate.and(predicate2).test("foo"); // true

Predicate<Boolean> nonNull = Objects::nonNull;
Predicate<Boolean> isNull = Objects::isNull;
Predicate<String> isEmpty = String::isEmpty;
Predicate<String> isNotEmpty = isEmpty.negate();
```

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione a valori booleani a un solo argomento generico T

All Methods | Static Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method and Description
default Predicate<T>	**and**(Predicate<? super T> other) Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.
static <T> Predicate<T>	**isEqual**(Object targetRef) Returns a predicate that tests if two arguments are equal according to **Objects.equals(Object, Object)**.
default Predicate<T>	**negate**() Returns a predicate that represents the logical negation of this predicate.
default Predicate<T>	**or**(Predicate<? super T> other) Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.
boolean	**test**(T t) Evaluates this predicate on the given argument.

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici

```
Function<String, Integer> toInteger = Integer::valueOf;
Integer k = toInteger.apply("123");
Function<Integer, Integer> square = k -> k*k;
Integer sqr = square.apply(5); // 25
```

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici

All Methods	Static Methods	Instance Methods	Abstract Methods	Default Methods

Modifier and Type	Method and Description
`default <V> Function<T,V>`	**`andThen`**`(Function<? super R,? extends V> after)` Returns a composed function that first applies this function to its input, and then applies the `after` function to the result.
`R`	**`apply`**`(T t)` Applies this function to the given argument.
`default <V> Function<V,R>`	**`compose`**`(Function<? super V,? extends T> before)` Returns a composed function that first applies the `before` function to its input, and then applies this function to the result.
`static <T> Function<T,T>`	**`identity`**`()` Returns a function that always returns its input argument.

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici

```
Function<String, Integer> toInteger = Integer::valueOf;
Function<Integer, String> toString = String::valueOf;
Function<String, String> backToString =
                        toInteger.andThen(toString);
backToString.apply("123"); // "123"
```



Esercizio

Creare una classe ElencoDiRoutine i cui oggetti sono costruiti con una lista di funzioni da stringa a intero da eseguire in sequenza una dopo l'altra

- Queste funzioni sono implementazioni dell'interfaccia java.util.function.Function
- La classe dispone inoltre del metodo esegui che prende in input un parametro di tipo stringa ed esegue le funzioni in sequenza stampandone l'output
- Costruire quindi un'istanza della classe con le seguenti funzioni (usando riferimenti a metodi laddove possibile):
  1. data in input una stringa x, restituisce la lunghezza di x
  2. data una stringa x, restituisce il numero di occorrenze del carattere 'y'
  3. data una stringa x, restituisce il corrispondente intero (assumendo che x contenga solo cifre)
  4. data una stringa x, restituisce la somma dei caratteri contenuti nella stringa

Interfacce funzionali built-in (java.util.function.*)

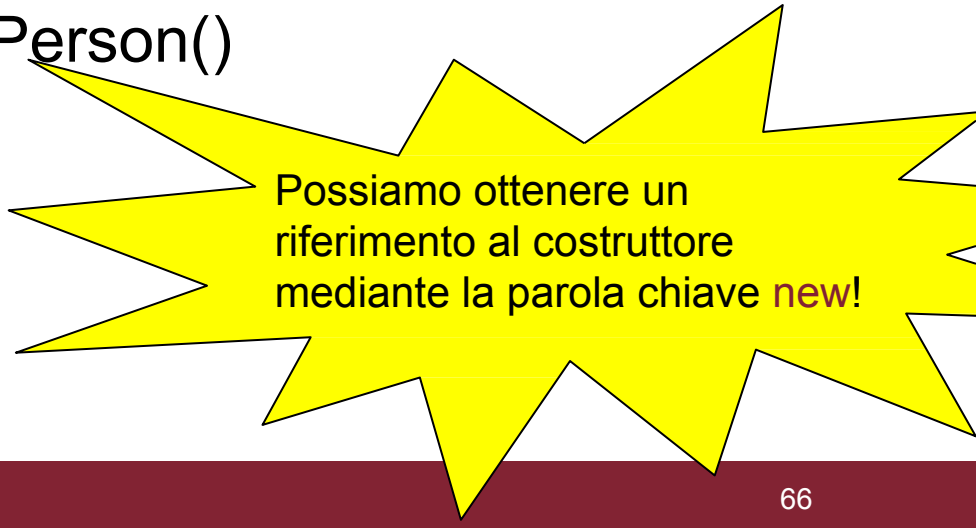
- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici
- Supplier<T>: funzione senza argomenti in input

```
Supplier<String> stringSupplier = () -> "ciao";
```

Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici
- Supplier<T>: funzione senza argomenti in input

```
Supplier<Person> personSupplier = Person::new;
personSupplier.get(); // new Person()
```



Interfacce funzionali built-in (java.util.function.*)

- Predicate<T>: funzione booleana a un solo argomento generico T
- Function<T,R>: funzione a un argomento di tipo T e un tipo di ritorno R entrambi generici
- Supplier<T>: funzione senza argomenti in input e un tipo di ritorno T generico
- Consumer<T>: funzione con un argomento di tipo generico T e nessun tipo di ritorno

```
Consumer<Person> greeter1 = p -> System.out.println("Hello, " + p.firstName);
greeter1.accept(new Person("Luke", "Skywalker"));
Consumer<Person> greeter2 = System.out::println;
```

Confronto tra riferimenti ed espressioni lambda

Riferimento a metodo	Tipo di metodo riferito	Espressione lambda equivalente
String::valueOf	Statico	x -> String.valueOf(x)
Object::toString	D'istanza, senza fissare l'istanza (intestazione estesa)	x -> x.toString()
x::toString	D'istanza, fissata l'istanza	() -> x.toString()
ArrayList::new	Costruttore	() -> new ArrayList<>()

For each su java.util.Collection

- Le collection sono ora dotate di un metodo forEach che prende in input un'interfaccia Consumer<? super T> dove T è il tipo generico della collection
- Ad esempio:

```
Collection<String> c = Arrays.asList("aa", "bb", "cc");

// In Java 7:
for (String s : c) System.out.println(s);

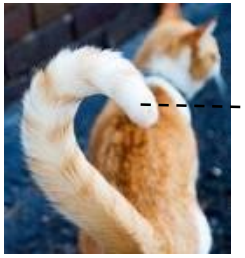
// In Java 8:
c.forEach(s -> System.out.println(s));
// persino meglio:
c.forEach(System.out::println);
```

🎯Obiettivi

- ✅ Java Collections Framework
- ✅ iterazione su Collection
- ✅ ADT & Collection Zoo: *List, *Set, *Map
- ✅ Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
  - Riapplichiamo ADT & Collection: Pila e Coda
  - Scegliere una struttura dati
  - Alberi

Pila e coda

- Due strutture dati fondamentali utili in un gran numero di attività
- Coda (FIFO: First In, First Out)



Coda (di gatto…)



Coda FIFO

- Pila o stack (LIFO: Last In, First Out)



Set



Stack LIFO

Coda



- Esempi di coda:
  - Coda degli eventi relativi a mouse e tastiera
  - Coda di stampa
- Esistono implementazioni standard della coda mediante l'interfaccia Queue
  - LinkedList implementa l'interfaccia Queue!
- Operazioni principali:
  - add: inserisce un elemento in coda
  - remove: rimuove un elemento dall'inizio della coda
  - peek: restituisce l'elemento all'inizio della coda senza rimuoverlo

Pila



- Esempi di pila:
  - La pila di esecuzione (run-time stack) contenente i record di attivazione delle chiamate a metodi
  - Nell'implementazione della ricorsione…
- Esiste un'implementazione standard mediante la classe Stack
  - Implementa l'interfaccia List
- Operazioni principali:
  - push: inserisce un elemento in cima alla pila
  - pop: rimuove l'elemento in cima alla pila
  - peek: restituisce l'elemento in cima alla pila senza rimuoverlo



Esercizio: parentesi annidate

- Scrivere un metodo che, data una stringa in input, verifichi se le parentesi della stringa sono correttamente annidate
  - Sono ammessi due tipi di parentesi: tonde e quadre



Esercizio: parentesi annidate

- Scrivere un metodo che, data una stringa in input, verifichi se le parentesi della stringa sono correttamente annidate
  - Sono ammessi due tipi di parentesi: tonde e quadre

```
import java.util.Stack;

public class Parentesi
{
    public boolean annidate(String s)
    {
        Stack<Character> p = new Stack<Character>();

        for (int k = 0; k < s.length(); k++)
        {
            char c = s.charAt(k);









        }

        return p.isEmpty();
    }
}
```



Esercizio: parentesi annidate

- Scrivere un metodo che, data una stringa in input, verifichi se le parentesi della stringa sono correttamente annidate
  - Sono ammessi due tipi di parentesi: tonde e quadre

```
import java.util.Stack;

public class Parentesi
{
    public boolean annidate(String s)
    {
        Stack<Character> p = new Stack<Character>();

        for (int k = 0; k < s.length(); k++)
        {
            char c = s.charAt(k);
            switch(c)
            {
                case '(': case '[': p.push(c); break;
                case ')': if (p.isEmpty() || p.pop() != '(') return false; break;
                case ']': if (p.isEmpty() || p.pop() != '[') return false; break;
            }
        }

        return p.isEmpty();
    }
}
```



Esercizio: inversione degli elementi

- Scrivere un metodo che, data una coda di interi in input, ne inverta gli elementi
  - Scrivere la variante in cui viene fornita in input (e restituita in output) una stringa



Esercizio: inversione degli elementi

- Scrivere un metodo che, data una coda di interi in input, ne inverta gli elementi
  - Scrivere la variante in cui viene fornita in input (e restituita in output) una stringa

```
public void inverti(Queue<Integer> q)
{
    Stack<Integer> s = new Stack<Integer>();

    while(!q.isEmpty())
    {
        Integer e = q.remove();
        s.push(e);
    }

    while(!s.isEmpty()) q.add(s.pop());
}
```



Esercizio: calcolatrice in notazione polacca inversa

- Scrivere un metodo che, data una lista in input con operandi e operatori in notazione polacca inversa, calcoli il risultato delle operazioni
  - Notazione polacca inversa: gli operatori vengono **DOPO** gli operandi
    (ad esempio: 3 4 + 5 - denota 3+4-5)



Esercizio: calcolatrice in notazione polacca inversa

- Scrivere un metodo che, data una lista in input con operandi e operatori in notazione polacca inversa, calcoli il risultato delle operazioni
  - Notazione polacca inversa: gli operatori vengono **DOPO** gli operandi
    (ad esempio: 3 4 + 5 - denota 3+4-5)

```
public int calcola(List<String> q) throws CalcolatriceException
{
    Stack<Integer> p = new Stack<Integer>();

    for (String v : q)
    {
        if (v.equals("+"))
            p.push(p.pop()+p.pop());
        else if (v.equals("-"))
        {
            Integer arg2 = p.pop();
            p.push(p.pop()-arg2);
        }
        else
        {
            Integer i = Integer.parseInt(v);
            p.push(i);
        }
    }

    // e' rimasto solo il risultato sulla pila?
    if (p.size() == 1) return p.pop();
    else
    {
        // la pila e' ancora piena
        throw new CalcolatriceException();
    }
}
```

🎯Obiettivi

- ✅ Java Collections Framework
- ✅ iterazione su Collection
- ✅ ADT & Collection Zoo: *List, *Set, *Map
- ✅ Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
- ✅ Riapplichiamo ADT & Collection: Pila e Coda
  - Scegliere una struttura dati
  - Alberi

Scegliere un contenitore

- Avete tutte queste strutture dati e dovete scegliere in quale struttura memorizzare i vostri oggetti
  - Che cosa fate?

1. stracciate tutto

2. capite a cosa serve ciascuna collezione!

Alcune domande utili (1)

- Come voglio accedere agli elementi?
  - Tramite una posizione? ArrayList…
  - Tramite una chiave? una mappa…
  - Non importa: un insieme…
- Quali sono i tipi degli elementi o tipi di chiavi e valori?
  - Ad esempio, in una mappa, vado da stringhe a interi o da interi a stringhe? Qual è la chiave univoca che deve determinare il valore?
- L'ordine degli elementi o delle chiavi è importante?
  - Sì, in un ordine naturale degli elementi: TreeMap o TreeSet
  - Sì, nell'ordine di inserimento: ArrayList, LinkedList, LinkedHashSet, LinkedHashMap
  - Non importa: HashSet o HashMap

Alcune domande utili (2)

- Per una collezione non mappa, quali operazioni devono essere veloci?
  - Ricerca veloce: HashSet
  - Aggiunta e rimozione di elementi: LinkedList
  - Non importa/ho pochi elementi nella collezione: ArrayList
- Se usate rappresentazioni ad albero (TreeSet o TreeMap), serve che si implementi Comparable?
  - No, se l'ordine naturale è già prestabilito (es. tipi primitivi)
  - Sì, è un tipo nuovo per il quale vogliamo stabilire l'ordinamento

Una coda non è sempre una coda!

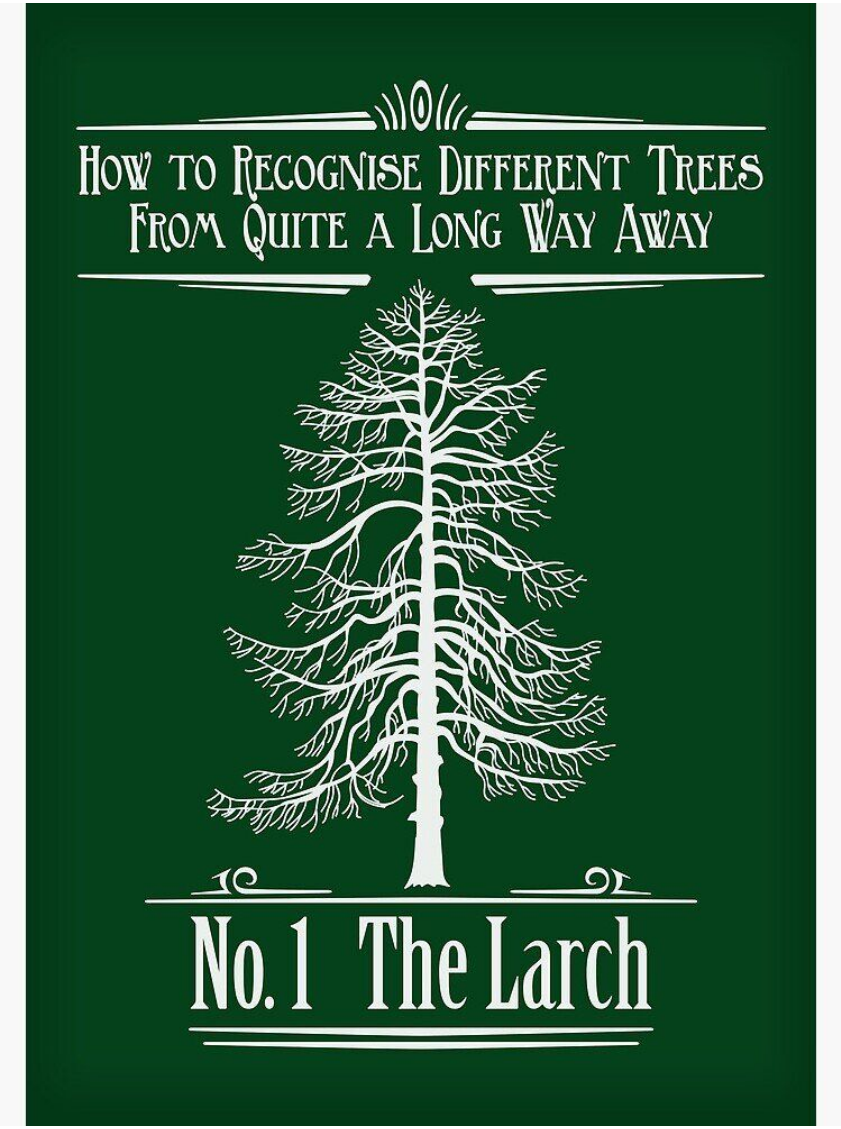
- Scegli la struttura dati sulla base dell'organizzazione dei dati



🎯Obiettivi

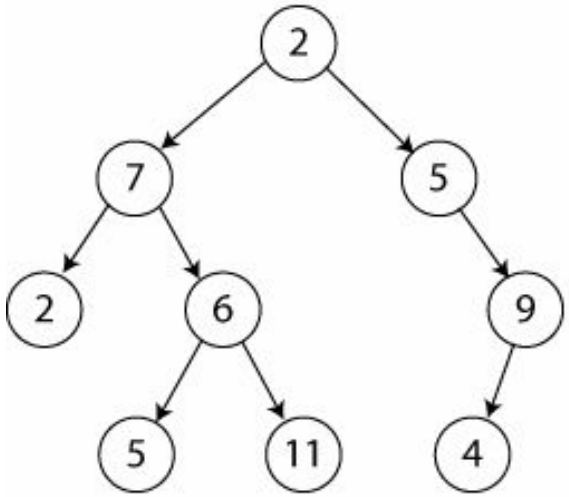
- ✅ Java Collections Framework
- ✅ iterazione su Collection
- ✅ ADT & Collection Zoo: *List, *Set, *Map
- ✅ Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
- ✅ Riapplichiamo ADT & Collection: Pila e Coda
- ✅ Scegliere una struttura dati
  - Alberi

Gli alberi


How to Recognise Different Trees
From Quite a Long Way Away
No. 1 The Larch

Gli alberi

- Una struttura dati ricorsiva in cui ogni nodo possiede un padre tranne la radice
- Gli alberi più comuni sono binari (ovvero con al più due figli per nodo)



Rappresentare gli alberi in Java

- Utilizziamo una classe annidata (interna se serve il riferimento all'albero) per rappresentare il nodo di un albero binario:

```
public class BinaryTree
{
    /**
     * Radice dell'albero
     */
    private Nodo root;

    static public class Nodo
    {
        private Nodo left;
        private Nodo right;
        private int valore;

        public Nodo(Nodo left, Nodo right, int valore)
        {
            this.left = left; this.right = right; this.valore = valore;
        }
    }
```

Ricerca all'interno di un albero binario

- Come ricercare un'informazione all'interno di un albero binario?

```
public class BinaryTree
{
    /**
     * Radice dell'albero
     */
    private Nodo root;

    static public class Nodo
    {
        private Nodo left;
        private Nodo right;
        private int valore;

        public Nodo(Nodo left, Nodo right, int valore)
        {
            this.left = left; this.right = right; this.valore = valore;
        }
    }

    public boolean contains(int val)
    {
        return contains(root, val);
    }

    private boolean contains(Nodo n, int val)
    {
        if (n == null) return false;
        if (n.valore == val) return true;
        return contains(n.left, val) || contains(n.right, val);
    }
}
```



Esercizio: Albero

- Partendo dall'esempio sugli alberi binari delle diapositive precedenti, scrivere una classe Albero che rappresenta un albero n-ario (con un numero variabile di figli per ciascun nodo) i cui nodi contengono un valore intero
- La classe espone il metodo contains che, dato in input un intero, restituisce true se un nodo dell'albero contiene l'intero, false altrimenti
- Sono implementati due meccanismi per aggiungere nodi all'albero:
  - Mediante classe annidata Nodo, che espone un metodo add il quale aggiunge il nodo in input tra i propri figli
  - Mediante un metodo Albero.add(intero1, intero2, …, interon) che aggiunge il cammino degli interi all'albero, verificando che il primo intero corrisponda con quello della radice e sollevando eccezione altrimenti; per gli altri interi, se non esistono vengono aggiunti come nodi intermedi
- La classe Albero espone il metodo remove che, dato in input un intero, rimuove il nodo corrispondente dall'albero aggiungendo tutti i suoi figli come figli del padre; nel caso il nodo fosse la radice, il metodo prende il suo primo figlio e lo sostituisce alla radice stessa

Altre strutture dati?

- Ci sono due importanti aggiunte al JDK Collections Framework
- Guava: libreria sviluppata principalmente da Google
- Apache collections: libreria del framework Apache
- Meno necessarie da Java 8, dati gli stream, che vedremo

🎯Obiettivi

- ✅ Java Collections Framework
- ✅ iterazione su Collection
- ✅ ADT & Collection Zoo: *List, *Set, *Map
- ✅ Manipolare collezioni: Riferimenti a metodi, Comparator, Interfacce funzionali built-in
- ✅ Riapplichiamo ADT & Collection: Pila e Coda
- ✅ Scegliere una struttura dati
- ✅ Alberi

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.